

Neovimのinit.lua入門

init.luaとLua設定の基礎

Neovimは設定ファイルとして従来の `init.vim` だけでなく、Luaで記述する `init.lua` もサポートしています。ただし両者を同時に使うことはできないため、Luaで設定を行う場合は `init.lua` (単独) を使用してください¹。`init.lua` ファイルはNeovimの設定ディレクトリ (`~/.config/nvim/` など) に配置します²。なお、`init.lua` 内で `vim.cmd` を使えば従来のExコマンド (Vimscript) を実行でき、逆に `init.vim` 内でLuaコードを実行することも可能です¹。既存のvimrcから移行する場合はファイル名を `init.lua` に変更し、必要に応じてLua文法に書き換えることでNeovimはLua設定を読み込むようになります。

また、`init.lua`以外にもNeovim起動時に自動実行したいLuaスクリプトがあれば、`runtimepath` 内の `plugin/` ディレクトリに置くことで自動で読み込ませることができます³。設定を分割して管理したい場合は、設定ディレクトリ内に `lua/` フォルダを作成し、そこにLuaモジュールファイルを配置して `require('モジュール名')` で読み込むことができます⁴。例えば `~/.config/nvim/lua/mysettings.lua` を用意すれば、`init.lua`から `require('mysettings')` として読み込めます⁵。`require()` は一度ロードしたモジュールをキャッシュするため、再度呼び出してもファイルは再実行されません。設定ファイルを再読み込みして変更を反映させたい場合は、一度 `package.loaded['<モジュール名>'] = nil` でキャッシュをクリアしてから `require()` を呼ぶ必要があります⁶。

オプションの設定

Neovimの各種オプションはLuaから簡単に設定できます。基本的には `vim.opt` を使ってオプション名に値を代入することで、Vimscriptの `:set` コマンドと同様の効果が得られます⁷。例えば `:set number` (行番号表示をオン) に相当する設定はLuaでは `vim.opt.number = true` となり、`:set expandtab!` (expandtabの切替) は `vim.opt.expandtab = false` のようにBoolean値を指定します⁸。同様に `:set relativenumber` は `vim.opt.relativenumber = true` のように設定できます。

`vim.opt` ではリスト型・マップ型のオプションもLuaのテーブルとして設定できます。例えば、`set wildignore=*.o,*.a,__pycache__` に対しては次のようにLuaテーブルで指定できます⁹：

```
vim.opt.wildignore = { '*.o', '*.a', '__pycache__' }
```

同様に、`set listchars=space:_,tab:>~` は：

```
vim.opt.listchars = { space = ' ', tab = '>~' }
```

`set formatoptions=njt` は：

```
vim.opt.formatoptions = { n = true, j = true, t = true }
```

のようにキーと真偽値のテーブルで設定できます⁹。

さらに、リスト型オプションには `:set+=` や `:set-=` に相当するメソッドも用意されています。例えば `vim.opt.shortmess:append({ I = true })` とすればショートメッセージオプションに `'I'` フラグを追加 (`:set shortmess+=I` 相当) でき、`vim.opt.whichwrap:remove({ 'b', 's' })` のように指定要素を除去することも可能です ¹⁰。

別の手段として、`vim.o` / `vim.bo` / `vim.wo` といったインターフェースでオプション値を直接操作する方法もあります ¹¹。`vim.o` はグローバルオプション、`vim.bo` はバッファローカルオプション、`vim.wo` はウィンドウローカルオプションに対応します ¹¹。例えば `vim.o.listchars = 'space:_,tab:>~'` のように文字列で一括設定したり、`vim.bo.shiftwidth = 4` として現在のバッファのshiftwidthを設定できます ¹²。また、`vim.bo[4].expandtab = true` のようにインデックスでバッファ番号を指定すれば特定バッファに対して設定することも可能です ¹³。通常の用途では `vim.opt` でほとんど事足りますが、Luaスクリプト内でオプション値を取得したい場合や特定バッファ・ウィンドウに対して直接設定を適用したい場合に、`vim.o` / `vim.bo` / `vim.wo` を使い分けることができます。

変数の利用

Vimscriptで使用する各種**変数スコープ**（グローバル `g:`、バッファ `b:`、ウィンドウ `w:`、タブページ `t:`、組み込み `v:`）に対応したLuaのテーブルが用意されています ¹⁴。具体的には以下のとおりです：

- `vim.g` : グローバル変数 (`g:`) ¹⁵
- `vim.b` : バッファローカル変数 (`b:`) ¹⁵
- `vim.w` : ウィンドウローカル変数 (`w:`) ¹⁵
- `vim.t` : タブページローカル変数 (`t:`) ¹⁵
- `vim.v` : 組み込み変数 (`v:`) ¹⁶
- `vim.env` : 環境変数 (エディタ起動中の環境) ¹⁶

グローバル変数 (Vimscriptの `g:`) は `vim.g` にキーとしてアクセスします。例えば Vimscriptで `let g:myvar = 42` としていたものは Luaでは `vim.g.myvar = 42` と表記します。多くのプラグインはグローバル変数経由で設定を受け取るため、同様に `vim.g` 経由で値を代入すればプラグインの設定ができます。よく使われる例として、リーダーキー (<Leader>) の設定は `vim.g.mapleader = ' '` のようにグローバル変数 `mapleader` に値を設定します (スペースをリーダーキーに設定する例。マッピング定義よりも前に設定します)。

Luaから `vim.g` 経由でリストや辞書型の値を代入すると、自動的にVim側でも対応する型 (List/Dictionary) として扱われます (データ型の相互変換は自動) ¹⁷。例えば次のように Lua のテーブルを設定すると：

```
vim.g.example_dict = { key1 = 'value', key2 = 100 }
```

Vimscript側では `g:example_dict` が同じ内容のDictionaryとして保持され、Luaから取得しても同じテーブル構造が得られます ¹⁷。このように、LuaとVimscript間のデータは透過的に変換されるため、特別な処理を意識せず利用できます。

バッファローカル変数や**ウィンドウローカル変数**も同様で、`vim.b` や `vim.w` 経由でアクセスします ¹⁵。例えば `vim.b[1].mybufvar = 'hello'` とすると**バッファ1番**の `b:mybufvar` が設定されます ¹⁸。`vim.w[100].winvar = 99` のようにウィンドウIDを指定してウィンドウ変数を操作することも可能です。その他、`vim.t` はタブページ変数、`vim.v` はVimの組み込み変数 (例: `vim.v.errmsg`)、`vim.env` は環境変数の参照・設定に使えます ¹⁹。例えば `print(vim.env.PATH)` とすればNeovim起動中の `$PATH` の値を表示できます。

なお、一部の変数名にはLuaの識別子として直接記述できない文字（ハイフン `-` や `#` など）を含む場合があります。そのような場合は `vim.g['my#var-name'] = 123` のように名前を文字列リテラルとして指定してアクセスしてください。例えばプラグインのautoload関数に対応する変数名（`g:plugin#name` など）はこの方法で設定できます。

キーマッピングの設定

Vimのキーマッピング（キーバインド）もLuaから設定できます。推奨される方法は `vim.keymap.set()` 関数を使うやり方です²⁰。この関数は少なくとも3つの引数を取り、第一引数に**モード**（文字列で `'n'`・`'i'`・`'v'`・`'c'`・`'t'` 等を指定。複数モードに同じマッピングを適用する場合は `{'n','v'}` のようにテーブルで指定）、第二引数に**押すキーのシーケンス**（lhs）、第三引数に**実行する内容**（rhs）を指定します²¹。

例えばノーマルモードで `<Leader>n` を押したときに行番号表示のON/OFFを切り替えるマッピングは以下のよう設定できます：

```
vim.keymap.set('n', '<Leader>n', '<cmd>set number!<CR>', { desc = 'Toggle line numbers' })
```

ここではrhsを文字列（Exコマンド）で指定し、`<cmd>...<CR>` という特殊表記によって `:set number!` コマンドを実行しています（通常の `:<Cmd>` マッピングに相当）。

次に、Lua関数をrhsに指定する例です。ノーマルモードで `<Leader>m` を押したらメッセージを表示するようにしてみます：

```
vim.keymap.set('n', '<Leader>m', function() print("Hello from Lua!") end, { silent = true })
```

このように、コマンド文字列の代わりに匿名関数を渡すことで任意のLuaコードを実行するマッピングを定義できます²²。また、複数のモードに同じマッピングを適用したい場合はモード引数にテーブル（例：`{'n','c'}`）を渡すことでノーマルモードとコマンドラインモードの両方に同じキーを割り当てる、といった指定も可能です²²。

第四引数のオプションテーブルでは、マッピングの挙動に関する細かな設定ができます。主なオプションは次の通りです：

- `buffer = true` : マッピングを**バッファローカル**に設定します（`{ buffer = 0 }` やバッファ番号を指定すると特定バッファのみ適用）²³。
- `silent = true` : マッピング実行時のメッセージ出力を抑制します²⁴。
- `expr = true` : **式マッピング**として定義し、rhsを実行せずその戻り値をキー入力として使用します²⁵。このオプションを付けると、Lua関数が文字列を返すようにして動的なマッピングを実現できます。
- `desc = '説明文'` : マッピングの説明を設定します。こうしておくと `:map` コマンドの一覧に表示された際に内容がわかりやすくなります²⁶（Lua関数を割り当てた場合でも説明が表示されます）。
- `remap = true` : **再帰的マッピング**を許可します。デフォルトではLuaで定義したマッピングは非再帰（`noremap` 相当）ですが、このオプションを `true` にすると他のマッピングを呼び出すキーも再帰的に機能するようになります²⁷。

自動コマンドの設定

自動コマンド (autocommand) は、特定のイベントが発生した際に自動的に実行されるコマンドや関数です²⁸。例えばファイルを開いたときや保存したとき、バッファ・ウィンドウを切り替えたとき、Insertモードに入った/抜けたときなど、さまざまなタイミングをフックして処理を挿入できます。

Luaでは `vim.api.nvim_create_autocmd(<イベント>, <オプションテーブル>)` を呼び出すことでオートコマンドを定義します²⁹。第一引数のイベントは文字列でイベント名を指定します (複数のイベントで共通の動作をさせたい場合、`{ 'BufEnter', 'BufWinEnter' }` のようにテーブルで複数指定も可能)²⁹。第二引数は設定項目を持つテーブルです。

主なイベント例:

- `BufRead` / `BufNewFile` : 既存ファイルを開いた、または新規ファイルを作成して開いたタイミング。
- `BufWritePre` / `BufWritePost` : バッファを保存する直前 / 保存直後。
- `InsertEnter` / `InsertLeave` : 挿入モードに入った直後 / 挿入モードを終了した直後。
- `BufEnter` / `BufLeave` : あるバッファに切り替わって入った時 / 別のバッファに移って離れた時。
- `FileType` : バッファの `filetype` (ファイルタイプ) が設定された時³⁰。通常ファイルを開いた際に自動で一度だけ発火し、そのバッファのファイルタイプ確定後に実行されます³⁰。

(この他にも多数のイベントがあります。詳細は `:help autocmd-events` を参照してください。)

オートコマンド作成時に指定するオプションテーブル内で、特に重要なキーは次の通りです³¹:

- `pattern` : パターン文字列 (またはそのリスト) です。イベントが発火した対象に対し、このパターンにマッチする場合のみ指定した処理を実行します³¹。たとえばイベントが `FileType` の場合、`pattern` には対象とするファイルタイプ名 (例: `"markdown"`) を指定します。イベントが `BufRead` 等の場合はファイル名パターン (例: `"*.txt"`) を指定します。省略した場合は `'*'` (すべてにマッチ) と見なされます³²。
- `callback` : イベント発生時に呼び出す **Lua関数** を指定します³³。関数内では必要に応じて `vim.api` 経由で各種操作を行います (例: オプション設定、関数呼び出しなど)。
- `command` : イベント発生時に実行する **Vimコマンド** を文字列で指定します³³。上記の `callback` とは排他的で、`callback` か `command` のどちらか一方を指定します³²。
- `group` : オートコマンドグループ名またはIDを指定します。このキーを付与すると、該当グループに属するオートコマンドとして登録されます (グループを利用することで設定の整理や重複実行の防止が可能です、後述)。
- `desc` : オートコマンドの説明を記述できます。これは任意ですが、付けておくと `:autocmd` の一覧表示で内容が分かりやすくなります。

オートコマンドグループを使うことで、関連する複数のオートコマンドをまとめたり、設定ファイル再読み込み時に重複登録されるのを防ぐことができます³⁴。Luaでは

`vim.api.nvim_create_augroup('<グループ名>', { clear = true })` という関数でグループを作成します³⁵。`clear = true` を指定すると同名のグループが既に存在する場合、そのグループ内の既存オートコマンドをすべて削除してから新しく作成します (これは Vimscript の `augroup グループ名 | autocmd!` に相当します)³⁶。グループ作成関数はグループID (番号) を返しますが、以降の `nvim_create_autocmd` では `group` に **文字列のグループ名** を指定しても同じグループを参照できます³⁷。グループを使っておけば、`init.lua`を再ソースしたときに以前登録したオートコマンドが二重に実行される心配がありません³⁴。

使用例: Markdownファイルを開いたときにスペルチェックを有効化するオートコマンドを定義してみます:

```
vim.api.nvim_create_autocmd('FileType',{
  pattern = 'markdown',
  callback = function()
    vim.opt_local.spell = true
    vim.opt_local.spelllang = { 'en', 'cjk' }
  end,
  desc = 'MarkdownファイルでスペルチェックをON'
})
```

上記では `FileType` イベントでファイルタイプが "markdown" の場合に、現在のバッファの `spell` オプションを有効にし（スペルチェックON）、`spelllang` オプションを英語+日本語（CJK）に設定しています。`vim.opt_local` を使うことで、そのローカルバッファに対してのみオプションを適用しています（Vimscript の `:setlocal` に相当）。

なお、Luaのコールバック関数は引数に1つ受け取りを定義することでイベント情報のテーブルを取得できます³⁸。例えば上記の例を `callback = function(args) ... end` としておき、`args.buf` でバッファ番号³⁹、`args.match` でマッチしたパターン文字列³⁸、`args.file` でファイル名³⁹ といった情報にアクセスできます。必要に応じてそれらの値を利用し、イベント発生対象に対する処理を柔軟に記述できます。

ユーザー定義コマンドの作成

Neovimではカスタムの `Ex` コマンド（ユーザーコマンド）をLuaで定義することができます。`vim.api.nvim_create_user_command()` 関数を使用し、第一引数に**コマンド名**（必ず先頭大文字）、第二引数に**実行する内容**（Exコマンド文字列またはLua関数）、第三引数に**オプションテーブル**を渡します⁴⁰。コマンド名は組み込みコマンドと区別するため**必ず大文字**で開始する必要があります⁴⁰。

例えば、単にメッセージを表示するコマンド `:Test` を作成してみます:

```
vim.api.nvim_create_user_command('Test', 'echo "It works!"', {})
```

これでエディタ上で `:Test` と実行すると「It works!」というメッセージが表示されます（Luaから呼ぶ場合は `vim.cmd.Test()` のようにドット記法で呼び出すことも可能です）⁴¹。

第三引数のオプションテーブルではコマンドの詳細な属性を指定できます。例えば `desc = '説明'` と記述すればコマンドの説明文を登録できますし、`force = false` とすれば同名の既存コマンドがある場合に上書きしないようにできます⁴²。そのほか、コマンド引数を受け取る場合の `nargs` や補完関数 `complete` など様々な属性を指定可能です（詳しくは `:help :command` を参照してください）。なお、Lua関数をコマンドの実行内容として指定した場合、その関数には一つのテーブル引数が渡され、`args.fargs`（引数のリスト）や `args.range`（行範囲）など必要な情報を参照できます。こちらは高度な使い方になりますが、必要に応じてNeovimのAPIドキュメントも参照してください。

Vimscriptコマンドと関数の呼び出し

Luaによる設定では極力LuaネイティブのAPIで完結させるのが望ましいですが、既存のVimscriptコマンドや関数を使いたい場合もあります。そのような場合、`vim.cmd()` 関数と `vim.fn` を使用することでLuaから直接Vimscriptを呼び出すことが可能です。

- **Vimscriptコマンドの実行:** `vim.cmd("...")` のように引数にExコマンド文字列を渡すと、そのコマンドを実行します⁴³。特殊文字（バックスラッシュやパーセント記号など）を含む場合は適宜エスケープが必要です⁴⁴。複数のコマンドをまとめて実行したい場合は、`[[ .. ]]` で囲んだ長い文字列リテラルを使うことで改行を含むコマンド列を一度に渡せます⁴⁵。例えば:

```
vim.cmd([[
highlight MyGroup guibg=Blue guifg=White
set number
]])
```

このように書くと、ハイライトグループの定義とオプション設定の2つのコマンドをまとめて実行できます⁴⁶。これはLua版のヒアドキュメント的な使い方で、Luaの `init.lua` 内に直接Vimscriptコードを埋め込む方法と言えます⁴⁶。

- **Vimscript関数の実行:** `vim.fn` を使うとビルトインのVimscript関数やユーザー関数をLuaから呼び出せます。例えばVimscriptの関数 `printf("Hello from %s", "Lua")` をLuaで呼ぶには `vim.fn.printf("Hello from %s", "Lua")` と記述します⁴⁷。戻り値はLuaで取得でき、Luaのテーブルや数値を引数に渡せばVim側ではListやNumberとして受け取られます（この型変換は自動で行われます⁴⁷）。例えば `vim.fn.reverse({'a', 'b', 'c'})` のようにLuaテーブル（リスト）を渡せば、Vimscript側でそのリストを反転した結果がLuaのテーブルとして返ってきます⁴⁷。このように、多くの組み込み関数を含めVimscriptの機能にLuaからアクセスできます。

（補足）関数名や変数名に `#` などLuaの識別子として使えない文字が含まれる場合、例えばVimscriptの関数 `MyPlugin#Do` は `vim.fn['MyPlugin#Do']()` のようにブラケット表記で呼び出してください⁴⁸。変数についても同様で、`vim.g['my#var']` のようにしてアクセスできます。

以上、初心者向けにLuaでNeovimの設定を書くための基本事項をまとめました。ここで紹介した方法を活用すれば、従来のvimrcと同等の設定を `init.lua` 内で自在に記述できるようになるでしょう。Luaによる設定は可読性が高く拡張もしやすいため、ぜひ快適なNeovim環境構築に役立ててください。

1 3 5 6 7 8 9 10 11 12 13 14 17 18 20 22 23 24 25 26 27 28 29 30 31 34 35 36 37 38 40 41 42 43 45 46 47 48

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 lua.txt

file://file_00000000bcac6206b1ba72a9dad43eb8

30 vim doesn't reload vimrc with pathogen - Super User

<https://superuser.com/questions/949590/vim-doesnt-reload-vimrc-with-pathogen>